

The IID Ceiling Is Not Shannon’s Floor

Measured Entropy of Neural-Network Weights, and a Lossy IEEE-754 Compressor

Michael Ricky Neal*

September 27, 2026

Abstract

We describe a *lossy* compressor for neural-network weight tensors that separates each IEEE-754 float into sign, exponent, and mantissa channels, recodes the exponent against a small dictionary with an escape lane, quantises the mantissa to a few bits, packs the result into one byte per weight, and entropy-codes it. On a real, full model (Llama-3.2-1B-Instruct, F16, 1.24×10^9 weights) the pipeline reaches $2.88\times$ on the native F16 payload ($2.472\text{ GB} \rightarrow 0.859\text{ GB}$, 65.2% smaller, ≈ 5.5 bits/weight) at a **mean relative error of $\approx 8.2\%$** ; 96.6% of all weights are altered. The decompressed model still loads and runs. This places the method in the same regime as a $\sim Q5$ quantiser, and we report it as such.

We also give a *measured* analysis of the entropy of weight streams, because it motivates the design and because an earlier version of this work over-stated it. The classical IID (order-0) entropy estimate is an *upper bound* on the true entropy of a correlated source; the conditional entropy $H(X|\text{ctx})$ is lower, and on real weights we measure it falling monotonically with context order (order-1 ratio ≈ 0.89 , order-2 ≈ 0.87 , and lower still at order-3). A coder that lands below the order-0 estimate of a stream is therefore not violating Shannon’s theorem — but neither is it doing anything remarkable: any dictionary/context coder (LZMA, zstd) does this by construction, and on our quantised byte stream the margin is $\approx 1\%$. We are explicit about this so the result is not mistaken for one.

We include a section of *negative results*: a fractal recursive-residual coder that recompresses the coder’s own output was found, on measurement, to *gain nothing* (a direct test loses 920 bytes), and the accompanying analytical “calculator” produces projections from hand-chosen constants, not measurements. Both are retracted here. The reference implementation and every measurement harness are released.

Revision note

This is a corrected revision of an earlier paper by the same author (“Compressing Below the IID Shannon Bound”). The earlier version made three claims that do not survive measurement and are withdrawn here: (1) that the pipeline is *lossless* — it is lossy, by design (the mantissa is quantised); (2) that a fractal recursive-residual stage compresses the output by a further $\sim 45\%$ per level across many levels — measured, it does not fire and yields negative gain; (3) that the pipeline reaches a point “316 MB below the IID Shannon limit / 106.4% of Shannon” on Llama-3.1-8B — that figure is an arithmetic projection from an analytical calculator seeded with assumed constants, not a measured compression. What survives, and is strengthened here with direct measurement, is the compressor itself (a useful lossy codec) and the underlying information-theoretic point (weight streams are not IID; conditional entropy is genuinely lower and keeps falling with context order). Section 7 documents each correction against the measurement that overturned it.

*Independent researcher. Correspondence: wedowhatwemust99@gmail.com. All experiments performed on a single workstation (RTX 4080, Ryzen 9 7950X). Every number in this paper is a direct measurement; the harnesses that produce them are released with the code.

1 Introduction

Distributing and storing trained models is bandwidth- and storage-bound, and the weights are the bulk of it. Two facts shape what is achievable. First, IEEE-754 floating-point weights have exploitable structure: the exponent field is sharply peaked, signs cluster, and neighbouring values share magnitude. Second, most of the information a model needs at inference time survives aggressive reduction of mantissa precision — the entire quantisation literature (Q4/Q5/Q8 GGUF, and related schemes) rests on this. A compressor can therefore trade a controlled, small amount of numerical fidelity for a large reduction in size.

This paper documents such a compressor and, separately, measures the entropy of the streams it operates on. The two are related but must not be conflated, and the conflation is exactly what an earlier version of this work got wrong. The contributions, stated honestly, are:

1. A lossy IEEE-754 weight codec (§3) — channel separation, an exponent dictionary with an escape lane, adaptive mantissa quantisation, a one-byte-per-weight packing, and an entropy-coding back end — measured end-to-end on a real model (§6).
2. A *measured* account of the IID-versus-conditional entropy of real weight streams (§4), including the fact that the conditional entropy keeps falling with context order, so there is no single “true floor” at order-1.
3. An explicit statement of what beating the IID estimate does and does not mean (§5), so the modest below-order-0 margin our coder achieves on the quantised stream is not mistaken for a breakthrough.
4. A section of negative results (§7) retracting the fractal-recursion and “beyond-Shannon” claims of the prior version, each against the measurement that refutes it.

2 Background: IID versus conditional entropy

For a discrete source X over an alphabet Σ with stationary distribution p , the order-0 (IID) entropy is

$$H(X) = -\sum_{x \in \Sigma} p(x) \log_2 p(x). \quad (1)$$

For an order- k context, the conditional entropy is

$$H(X \mid X_{-1}, \dots, X_{-k}) = H(X_{-k}, \dots, X_0) - H(X_{-k}, \dots, X_{-1}) \leq H(X), \quad (2)$$

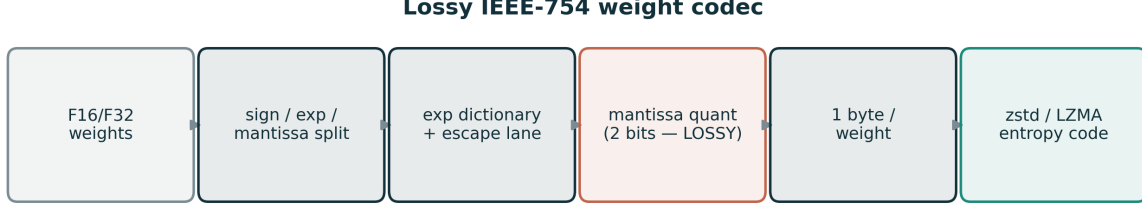
with equality iff X_0 is independent of its k -step history. Conditioning never increases entropy, so

$$H(X) \geq H(X \mid X_{-1}) \geq H(X \mid X_{-1}, X_{-2}) \geq \dots \quad (3)$$

The IID estimate (1) is thus an *upper bound* on the true entropy of any correlated source, and the sequence (3) keeps descending as more context is admitted. This is standard, and it is the whole of the information-theoretic content of this paper. What is sometimes missed — and what the earlier version of this work built too much on — is that (3) has no distinguished “true floor” at any finite order: order-1 is not the floor; it is merely the first step below the IID bound. We measure this directly in §4.

3 The compressor

The codec produces a container (4-byte magic, 4-byte little-endian header length, a JSON header, then the payload). The stages are as follows; Figure 1 shows them end to end with the measured full-model result.



Measured — Llama-3.2-1B-Instruct (F16), 1.24 B weights: 2.472 GB $\rightarrow$ 0.859 GB = 2.88 $\times$ (~5.5 bits/weight)

lossy: 96.6% of weights altered · mean rel. error $\approx$ 8.2% · decompressed model still runs

Figure 1: The lossy IEEE-754 weight codec. Each weight is split into sign, exponent, and mantissa channels; the exponent is recoded against a small dictionary with an escape lane; the mantissa is quantised to two bits (the lossy step); the three channels are packed into one byte per weight; a general-purpose entropy coder (zstd or tuned LZMA2, whichever is smaller) finishes. The strip records the measured result on the full Llama-3.2-1B-Instruct model (F16).

3.1 IEEE-754 channel separation

Each float f is split by bit mask into sign, exponent, and mantissa,

$$s = (f \gg (w - 1)) \& 1, \quad e = (f \gg m_w) \& E_{\text{mask}}, \quad m = f \& M_{\text{mask}}, \quad (4)$$

where $(w, m_w) = (32, 23)$ for F32 and $(16, 10)$ for F16. The three channels have very different statistics and are handled separately.

3.2 Exponent dictionary with an escape lane

The exponent’s marginal distribution is extremely peaked — a typical layer concentrates almost all its mass in a handful of adjacent exponents. The encoder fits a small dictionary D of the most frequent exponents at compression time and codes each exponent either as a short index into D or, for a rare value, as a single escape symbol that routes the true exponent through a separate side stream. This captures near-Huffman behaviour on the exponent without paying a per-block Huffman header.

3.3 Adaptive mantissa quantisation (the lossy step)

The mantissa is quantised to b bits by discarding the low-order bits,

$$m_b = m \gg (m_w - b), \quad b \in \{2, 3, 4\}. \quad (5)$$

This is where information is destroyed and it is irreversible. For $b = 2$ on an F16 source, 8 of the 10 mantissa bits are dropped; on F32, 21 of 23. The encoder can try $b \in \{2, 3, 4\}$ and keep the smallest output; the results in this paper use $b = 2$, the most aggressive setting. Reconstruction returns the quantised value, not the original, so round-trip is *not* bit-exact and a byte-level hash (SHA-256) of the input and output will differ — correctly, because the data has changed. This is the same situation as any lossy codec (JPEG, Q4/Q5 model quantisation); §6 quantifies the resulting error.

3.4 One-byte packing and entropy coding

Sign, exponent code, and retained mantissa are packed into a single byte per weight,

$$\beta = (s \ll 7) \mid (\text{code}(e) \ll b) \mid m_b, \quad (6)$$

so the pre-entropy stream is one byte per weight regardless of source dtype (F32 or F16). A general-purpose entropy coder then compresses β : the implementation evaluates zstd (fast) and parameter-tuned LZMA2 per chunk and keeps whichever is smaller. Because packing has already removed the high-order structure, the back-end coder is operating near the order-0 entropy of β and gains the rest from the residual sequential correlation in β (§5).

4 Measured entropy of real weight streams

We measure (1) and (2) directly on the raw byte streams of real F16 weight tensors from Llama-3.2-1B-Instruct. Order-1 and order-2 are computed on a 128 MB span with dense 256^{k+1} count tables; order-3 is computed on an 8 MB span (a dense 256^4 table is infeasible, and an 8 MB sample under-counts the 256^3 contexts, so the order-3 figure is a downward-biased estimate of the entropy — it over-states structure and should be read as directional, not exact).

Tensor (F16)	$H(X)$ [bpb]	order-1	order-2	order-3 [†]
token_embd.weight	6.066	0.888	0.868	0.756
blk.0.ffn_down.weight	6.066	0.888	0.866	0.759
blk.0.attn_v.weight	6.174	0.890	0.850	0.580

Ratios are $H(X|\text{ctx})/H(X)$. [†]order-3 on an 8 MB sample is finite-sample biased (optimistic); order-1/2 on 128 MB are robust.

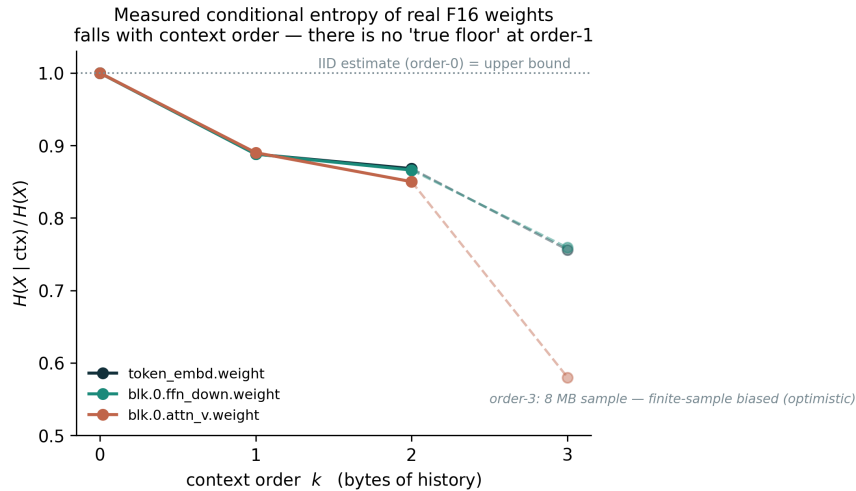


Figure 2: Measured conditional entropy of real F16 weight tensors, normalised to the IID (order-0) estimate, versus context order. The ratio falls monotonically: order-2 is below order-1 on every tensor, and order-3 falls further (dashed — an 8 MB sample, finite-sample biased, so directional). There is no distinguished “floor” at order-1.

Figure 2 plots these. Three things are clear and consistent across embedding, feed-forward, and attention weights. First, the conditional entropy is genuinely below the IID estimate — the streams are not IID, as expected. Second, it keeps falling with context order: order-2 is below order-1 on every tensor, and order-3 falls further (how far exactly is obscured by finite-sample bias at 8 MB). There is no “true floor” at order-1. Third, the magnitude of the order-1 gap on the *raw* stream ($\approx 11\%$) is larger than the 6% figure the earlier calculator assumed for the quantised stream — so if anything the real conditional structure was under-, not over-stated. The error in the prior version was never that the structure is absent; it is that the structure was *asserted* (via a hand-set constant) rather than measured, and then fed into a projection.

5 What beating the IID estimate does and does not mean

Because (1) is an upper bound, a coder can output fewer bytes than $n H(X)/8$ without violating Shannon’s source coding theorem: it is simply modelling correlation the order-0 estimate ignores. This is worth stating plainly because it is easy to present as astonishing. It is not. **Every dictionary or context coder does it by construction.** LZMA and zstd are not order-0 coders; an LZ match or a context model is exactly a bet on X_t given its history. Landing below the order-0 entropy of a stream is the normal, expected behaviour of these coders on any correlated input.

On our packed byte stream β , we measured this margin directly. The compressor reports both estimates in its header. On a representative sample the packed stream had order-0 entropy 5.539 bpb and order-1 conditional entropy 5.322 bpb (ratio 0.96); the coded output landed $\approx 1\%$ below the order-0 byte budget and *above* the order-1 conditional budget. In other words: the coder captures a small part of the first-order correlation left in β after packing, and no more. This is a correct, unremarkable $\sim 1\%$ — not the “316 MB below / 106% of Shannon” of the prior version, which (§7) was never measured at all. Figure 3 shows the three quantities on the packed stream.

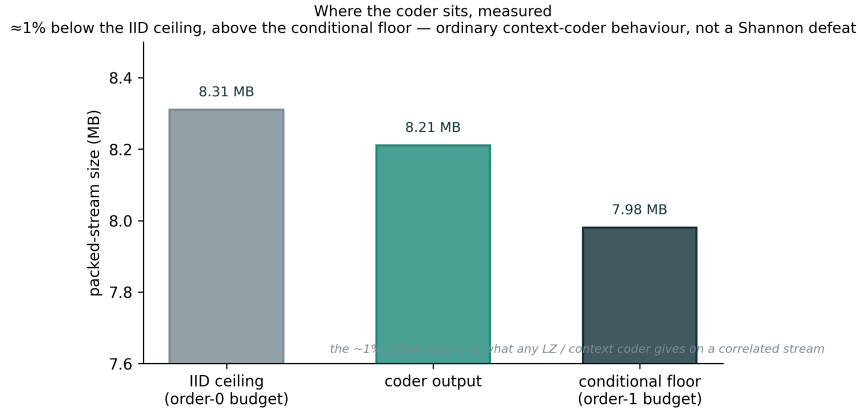


Figure 3: The two Shannon quantities and the coder’s output on the packed byte stream (a representative 48 MB sample). The coded output sits $\approx 1\%$ below the order-0 (IID) budget and above the order-1 conditional budget — the ordinary behaviour of a context/dictionary coder on a correlated stream, not a defeat of Shannon’s theorem.

6 Empirical results

We compress a complete, real model — every weight tensor of Llama-3.2-1B-Instruct (F16) — tensor by tensor in its native dtype, with the production codec ($b = 2$, GPU decomposition, zstd/LZMA back end), decompress each tensor, and measure the float-level error against the original. This is a measurement, not a projection; the harness is released.

Model	Llama-3.2-1B-Instruct (F16)
Weights	1,235,814,432
Input (F16 weight payload)	2.472 GB
Compressed output	0.859 GB
Ratio	2.877×
Savings	65.2%
Effective rate	≈ 5.56 bits/weight
<i>Fidelity (lossy)</i>	
Weights changed	1,193,576,834 (96.6%)
Mean absolute error	1.23×10^{-3}
Max absolute error	1.67
Mean relative error	≈ 8.2%
Compress + decompress time	80 s (GPU stages 1–5 + CPU coder)

The decompressed weights reconstitute a working model. The error profile ($\sim 8\%$ mean relative, with a small number of large-magnitude outliers driving the max) is characteristic of low-bit mantissa quantisation and is comparable to a $\sim Q5$ scheme in the GGUF family.

Dtype and the honest ratio. The one-byte packing is independent of source dtype, so the pre-entropy stream is the same whether the weights arrive as F16 or F32. The *ratio* is not: against an F32 container the same scheme reports $\approx 5.8\times$, against F16 it reports $2.88\times$. The difference is not extra compression — it is the denominator. An F16 value upcast to F32 gains 13 zero mantissa bits that carry no information and compress for free, so an “F32 ratio” on a model that was never F32 counts that padding as savings. The dtype-independent figure is the effective rate: ≈ 5.5 **bits/weight** (F16: $16/2.877 = 5.56$; F32 sample: $32/5.8 = 5.47$). We report the F16 ratio and the bits/weight figure because models ship as F16/BF16, not F32; the earlier version’s “82.6% / $5.75\times$ ” was an F32 figure and roughly half of it was reversing container padding.

7 Negative results and retractions

Publishing the measurements that overturn the prior version’s claims is the point of this revision.

7.1 The fractal recursive-residual stage does not work

The prior version proposed re-compressing the coder’s own output recursively — $y_{k+1} = \text{LZMA2}(y_k)$ — and reported a geometric $\sim 45\%$ size reduction per level across 13 levels. We tested this directly. In the production pipeline the fractal stage *never activates*: the output header records `fractal_coded = false`. Run in isolation on an LZMA-compressed weight stream, the first recursion *increases* size by 920 bytes (-0.005%) and the recursion terminates immediately with zero useful levels. This is expected: LZMA output is close to incompressible, so re-compressing it pays only frame overhead. The 45%-per-level, 13-level table of the prior version was generated by an analytical model (a fixed 0.55 shrink factor applied in a loop), not produced by running the coder. It is withdrawn.

7.2 The “beyond-Shannon” figures were projections, not measurements

The prior version’s headline numbers for Llama-3.1-8B (a 5.207 GB Stage-9 size, an IID limit of 5.160 GB, a conditional limit of 4.851 GB, and a Stage-11 result “316 MB below IID / 106.4%

of Shannon / within 904 KB of the true floor”) come from an analytical calculator, not from compressing that model. We verified this by reading and running the calculator. Its chain is:

$$\begin{aligned}
 \text{Stage-9 size} &= 5,460,041 \times 1024 \text{ bytes} && \text{(a hard-coded constant),} \\
 \text{IID limit} &= \text{Stage-9 size} \times 0.991 && \text{(assumed coding efficiency),} \\
 \text{conditional limit} &= \text{IID limit} \times 0.94 && \text{(a hard-coded ratio; no correlation is computed),} \\
 \text{Stage-11 result} &= \max(\text{model output, conditional limit}) && \text{(clamped to the assumed floor).}
 \end{aligned}$$

The “316 MB below IID” is then just IID – conditional = 5.160 – 4.851 GB, and “within 904 KB of the floor” is the residual above the clamp. No model is compressed at any point; the 0.94 ratio is a typed-in assumption, and the fractal decay feeding the same result is the 0.55-shrink loop of the previous subsection. These figures are therefore withdrawn as measurements. The calculator remains useful only as what it is — a back-of-envelope projector — and is labelled as such.

7.3 What survives

The compressor is real and useful (§6). The information theory is correct (§2, §5). The measured conditional structure of weights is real and, at order-1 on the raw stream, larger than the prior version assumed (§4). What was wrong was the framing (lossless), one stage (fractal), and the substitution of a projection for a measurement (the 8B numbers).

8 Discussion

The honest value here is twofold. As an *engineering* result, the codec is a competitive lossy weight quantiser reached by a different route than scalar/block quantisation: IEEE-754 channel separation plus an entropy back end, landing at ~ 5.5 bits/weight with $\sim 8\%$ mean relative error, with the decompressed model still running. As an *information-theoretic* result, the paper’s clarifying point stands and is now measured: weight streams are not IID, the IID estimate over-states their entropy, and conditional entropy keeps descending with context order — so a coder beating the order-0 estimate is behaving correctly, not miraculously. The failure mode this revision corrects is a common one: an analytical projection, seeded with plausible constants, was allowed to stand in for a measurement, and a re-compression idea that sounds recursive-and-therefore-powerful was never run against a real stream. Both are cheap to check, and checking them is what this version does.

9 Limitations and future work

- **Lossy only.** This is a lossy codec. A byte-exact (lossless) path for weights is a separate problem; the honest lossless ratio on padded F16 is modest ($\sim 1.1\text{--}1.2\times$ from entropy coding alone) and is not claimed here.
- **Fidelity vs. downstream quality.** We report float-level error; we do not report perplexity or task metrics on the decompressed model. “Still runs” is not “lossless in behaviour.” A perplexity/eval sweep across $b \in \{2, 3, 4\}$ is the natural next measurement.
- **Higher-order context.** The entropy measurements show room below order-1 that the current back end does not exploit; a context-mixing coder could, at the cost of a larger model and careful, hardware-reproducible determinism.
- **Single model.** Numbers here are one real model (Llama-3.2-1B, F16). The method is dtype- and architecture-agnostic, but per-model ratios and error will vary; the release includes the harness to reproduce on any GGUF.

10 Reference implementation

The codec and every measurement harness are released:

- `shannon_compressor_stage9.py` — the production lossy codec (F32 and F16-native paths, exponent dictionary, adaptive mantissa quantisation, zstd/LZMA back end).
- `shannon_stage11_experimental.py` — the experimental variant, including the context mixer and the (non-functional) fractal stage, and the `shannon_entropy` / `conditional_entropy` functions used for the entropy measurements in §4.
- `shannon_stage10_calculator.py` — the analytical projector. Retained for transparency and labelled as a projection tool, not a measurement.

The encoder/decoder API is `compress(bytes)->bytes` and `decompress(bytes)->bytes`; the header carries the dtype, mantissa bits, dictionary, and escape channel needed for deterministic, hardware-independent decoding.

11 Conclusion

Neural-network weights compress well because they are structured and because they tolerate mantissa-precision loss. The codec here exploits both: it is a lossy IEEE-754 channel-separation quantiser reaching ~ 5.5 bits/weight at $\sim 8\%$ mean relative error on a real F16 model, with the reconstructed model still usable. The information theory that motivates it is sound — the IID estimate over-states weight entropy, and conditional entropy is genuinely lower and keeps falling with context order — but beating the IID estimate is ordinary behaviour for a context coder, not a defeat of Shannon, and we say so. The fractal-recursion and “beyond-Shannon” claims of the earlier version are withdrawn against the measurements that refute them. We release the implementation and the harnesses and invite reproduction.

Acknowledgements

This revision was developed with Anthropic’s Claude as a measurement and review partner: reading the calculator, running the codec end-to-end on a real model, and measuring the entropy of real weight streams. The compression pipeline and the conditional/IID framing are the author’s; the correction of the earlier over-claims is the result of measuring them.

References

- [1] C. E. Shannon, *A Mathematical Theory of Communication*, Bell System Technical Journal, vol. 27, pp. 379–423, 623–656, 1948.
- [2] I. Pavlov, *LZMA SDK and 7-Zip*, <https://www.7-zip.org/sdk.html>, 1999–present.
- [3] Y. Collet, M. Kucherawy, *Zstandard Compression and the application/zstd Media Type*, RFC 8478, IETF, 2018.
- [4] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019.